Registrar workflow state machine & RBAC matrix: department draft, Academic Affairs, final publish, and role-based permissions

Computer
Role-based permission matrix & approval workflow: department schedulers, chairs, Academic Affairs, and Registrar roles
Computer
Show the RBAC permission matrix for these states
Map the API endpoints for this approval state machine
Show the database schema for the registrar workflow
Artifacts
18
Sources
358

Below is the complete registrar workflow view tying together the **state machine**, **RBAC by state**, **API surface**, and **database schema** for Semester's scheduling and approval console. Institutions commonly use designated department schedulers, chair/dean review, registrar coordination, and Academic Affairs or provost-level approval for exceptions such as non-standard meeting times.[1] [2] [3]

Workflow state machine

A realistic workflow separates standard approval from policy exceptions and central-room coordination. Registrar workflow practices and scheduling policies commonly put departments and colleges on the build path, then route special approvals through Academic Affairs or provost authority before final publication.[4] [2] [5]

State list

State	Description	Primary actor
`draft`	Department scheduler is building or editing section data	Department scheduler
`validation_pending`	System rules engine is evaluating completeness and conflicts	System
`validation_failed`	Blocking issues prevent advancement	Department scheduler
`department_ready`	Department believes record is ready for chair review	Department scheduler
`chair_review`	Department leadership approval stage	Chair / program director
`dean_review`	College-level review and prioritization	Dean / associate dean
`registrar_review`	Official schedule and policy review	Registrar scheduler
`room_coordination`	Central room assignment and resource resolution	Registrar / room manager
`academic_affairs_review`	Exception or policy-sensitive review	Academic Affairs
`approved_for_publish`	All approvals complete and publish checks passed	Registrar admin
`published`	Visible in official schedule	Registrar admin
`change_requested`	Published section change proposed	Department / registrar
`locked`	Official record locked except emergency change path	Registrar admin
`denied`	Exception or request rejected	Relevant approver
`archived`	Historical read-only state	System

Transition map

```
draft
  → validation_pending
  → archived

validation_pending
  → validation_failed
  → department_ready

validation_failed
  → draft
```

```
  → academic_affairs_review

department_ready
  → chair_review

chair_review
  → dean_review
  → draft
  → academic_affairs_review

dean_review
  → registrar_review
  → draft
  → academic_affairs_review

registrar_review
  → room_coordination
  → approved_for_publish
  → draft
  → academic_affairs_review

room_coordination
  → approved_for_publish
  → registrar_review
  → draft

academic_affairs_review
  → registrar_review
  → denied
  → draft

approved_for_publish
  → published
  → draft

published
  → change_requested
  → locked

change_requested
  → validation_pending
  → registrar_review

locked
  → archived
  → registrar_review   (emergency unlock path)

denied
  → archived
  → draft
```

Transition rules

- `draft → validation_pending` requires required fields, including term, course, section identity, capacity, and at least one meeting proposal or explicitly approved TBA status.[6] [7]
- `validation_pending → department_ready` requires no blocking conflicts and successful schedule-policy checks. Standard meeting pattern enforcement is a common registrar rule because it reduces conflicts and improves room use.[8] [9]
- `chair_review` and `dean_review` confirm academic need, staffing feasibility, and college priorities before central processing. Chair/dean review is explicitly part of published scheduling timelines at some institutions.[3]
- `registrar_review → academic_affairs_review` is required for non-standard scheduling, special approvals, or policy exceptions. Some institutions reserve non-standard meeting-time approval to provost/Academic Affairs authority.[2] [5]
- `room_coordination` is required when central-room inventory is involved or when conflicts remain. Registrar scheduling policies commonly centralize general-use classroom assignment.[1]
- `approved_for_publish → published` requires all approvals, resolved conflicts or logged waivers, and a publishable snapshot.
- `published → change_requested` is the controlled post-publication change path; institutions often restrict who can change published schedules and route those changes through workflow.[10] [11]

RBAC permission matrix by state

Education RBAC should distinguish view, edit, export, approval, and override authority, with access scoped to legitimate institutional purpose and logged for audit.[12] [13] [14]

Roles

Role	Typical scope
Department scheduler	Department or program
Chair / program director	Department or program
Dean / associate dean	College or school
Registrar scheduler	Institution, campus, or college
Registrar admin	Institution or campus
Academic Affairs reviewer	Institution or college
Room manager	Central pool or scoped room inventory
Auditor	Read-only scoped access

State-by-state matrix

State	Dept scheduler	Chair	Dean	Registrar scheduler	Registrar admin	Acade
`draft`	View/Edit/Submit	View	View	View/Edit	View/Edit	View
`validation_pending`	View	View	View	View	View	View
`validation_failed`	View/Edit/Resubmit	View/Comment	View/Comment	View/Comment	View/Comment	View
`department_ready`	View	View/Approve/Return	View	View	View	View
`chair_review`	View/Comment	View/Approve/Return	View	View	View	View
`dean_review`	View	View/Comment	View/Approve/Return	View	View	View
`registrar_review`	View	View	View	View/Approve/Return/Exception-route	View/Override	View
`room_coordination`	View/Comment	View	View	View/Assign/Return	View/Override	View
`academic_affairs_review`	View	View	View	View/Comment	View	View/Appro
`approved_for_publish`	View	View	View	View	View/Publish/Return	View
`published`	View/Request-change	View/Request-change	View/Request-change	View/Request-change	View/Lock/Unpublish-policy	View
`change_requested`	View/Edit if reopened	View/Comment	View/Comment	View/Approve/Return	View/Override	View
`locked`	View	View	View	View	View/Emergency-unlock	View
`denied`	View/Rework	View	View	View	View	View
`archived`	Scoped view	Scoped view	Scoped view	Scoped view/export	Scoped view/export	Scoped vie

Action matrix

Capability	Dept scheduler	Chair	Dean	Registrar scheduler	Registrar admin	Academic Affairs	Room manager
View department records	Yes	Yes	Yes	Yes	Yes	Scoped	Scoped
Edit draft fields	Yes	No	No	Yes	Yes	No	No
Submit to workflow	Yes	Yes	Yes	Yes	Yes	No	No
Approve at assigned stage	No	Yes	Yes	Yes	Yes	Yes	No
Return for correction	No	Yes	Yes	Yes	Yes	Yes	Yes, room issues only
Assign room	Dept pool only	No	No	Yes	Yes	No	Yes
Override conflict	No	No	No	Limited	Yes	Limited	Limited, room only
Export scoped data	Limited	Limited	Limited	Yes	Yes	Limited	Limited
Publish	No	No	No	No	Yes	No	No
Emergency unlock	No	No	No	No	Yes	No	No

API endpoints

The API should expose both record operations and workflow actions. Workflow systems in registrar contexts are typically built around routed requests, approvals, and review queues rather than simple CRUD alone.[10] [4]

Core resources

```
/api/academic-years
/api/terms
/api/parts-of-term
/api/courses
/api/sections
/api/section-meetings
/api/rooms
/api/room-features
/api/workflow-instances
/api/workflow-tasks
/api/conflicts
/api/audit-events
```

Term and calendar endpoints

```
GET    /api/terms
POST   /api/terms
GET    /api/terms/{termId}
PATCH  /api/terms/{termId}
POST   /api/terms/{termId}/publish-window
GET    /api/terms/{termId}/calendar-events
POST   /api/terms/{termId}/calendar-events
PATCH  /api/calendar-events/{eventId}
POST   /api/calendar-events/{eventId}/submit
POST   /api/calendar-events/{eventId}/approve
POST   /api/calendar-events/{eventId}/reject
```

Section scheduling endpoints

```
GET    /api/sections?termId=&orgUnitId=&state=
POST   /api/sections
GET    /api/sections/{sectionId}
PATCH  /api/sections/{sectionId}
DELETE /api/sections/{sectionId}
POST   /api/sections/{sectionId}/copy-from-prior-term
POST   /api/sections/{sectionId}/submit
POST   /api/sections/{sectionId}/reopen
POST   /api/sections/{sectionId}/request-change
POST   /api/sections/{sectionId}/lock
POST   /api/sections/{sectionId}/unlock-emergency
```

Workflow action endpoints

```
GET    /api/workflow-tasks?assignee=me&state=open
POST   /api/workflow-tasks/{taskId}/approve
POST   /api/workflow-tasks/{taskId}/return
POST   /api/workflow-tasks/{taskId}/deny
POST   /api/workflow-tasks/{taskId}/route-academic-affairs
POST   /api/workflow-tasks/{taskId}/comment
```

Validation and conflict endpoints

```
POST   /api/sections/{sectionId}/validate
GET    /api/sections/{sectionId}/conflicts
GET    /api/conflicts?termId=&orgUnitId=&severity=
POST   /api/conflicts/{conflictId}/waive
POST   /api/conflicts/{conflictId}/resolve
GET    /api/rooms/availability?termId=&day=&start=&end=&capacity=
POST   /api/sections/{sectionId}/assign-room
```

Publish endpoints

```
POST   /api/terms/{termId}/prepublish-check
GET    /api/terms/{termId}/publication-preview
POST   /api/terms/{termId}/publish
```

```
POST   /api/terms/{termId}/enter-refine-mode
POST   /api/terms/{termId}/lock
```

Audit and access endpoints

```
GET    /api/audit-events?targetType=section&targetId=
GET    /api/audit-events?actorId=&module=registrar
GET    /api/access/effective?userId=&resourceType=&resourceId=
POST   /api/exports/sections
```

Every approval, export, override, and role-sensitive action should emit an audit event with actor, scope, action, and reason. Guidance on RBAC and FERPA-oriented systems emphasizes logging exports, overrides, and permission-sensitive operations.[13] [12]

Database schema

The workflow schema should separate academic data from approval workflow, conflict detection, and access control. That keeps official schedule records stable while workflow actions remain append-only and auditable.

Academic structure

```
CREATE TABLE academic_years (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  status text NOT NULL CHECK (status IN ('draft','active','archived'))
);

CREATE TABLE terms (
  id uuid PRIMARY KEY,
  academic_year_id uuid NOT NULL REFERENCES academic_years(id),
  code text UNIQUE NOT NULL,
  name text NOT NULL,
  starts_on date NOT NULL,
  ends_on date NOT NULL,
  state text NOT NULL CHECK (
    state IN (
      'draft','validation_pending','validation_failed','department_ready',
      'chair_review','dean_review','registrar_review','room_coordination',
      'academic_affairs_review','approved_for_publish','published',
      'change_requested','locked','denied','archived'
    )
  ),
  publish_version integer NOT NULL DEFAULT 0
);

CREATE TABLE courses (
  id uuid PRIMARY KEY,
  subject_code text NOT NULL,
  catalog_number text NOT NULL,
  title text NOT NULL,
  credit_hours numeric(4,2) NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  UNIQUE(subject_code, catalog_number)
);

CREATE TABLE sections (
  id uuid PRIMARY KEY,
  course_id uuid NOT NULL REFERENCES courses(id),
  term_id uuid NOT NULL REFERENCES terms(id),
  section_number text NOT NULL,
  owning_org_unit_id uuid NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  workflow_state text NOT NULL CHECK (
    workflow_state IN (
      'draft','validation_pending','validation_failed','department_ready',
      'chair_review','dean_review','registrar_review','room_coordination',
      'academic_affairs_review','approved_for_publish','published',
      'change_requested','locked','denied','archived'
    )
  ),
  last_submitted_at timestamptz,
  published_at timestamptz,
  locked_at timestamptz,
  UNIQUE(course_id, term_id, section_number)
);

CREATE TABLE rooms (
```

```
  id uuid PRIMARY KEY,
  building_code text NOT NULL,
  room_code text NOT NULL,
  capacity integer NOT NULL CHECK (capacity > 0),
  room_pool_type text NOT NULL CHECK (room_pool_type IN ('central','department')),
  owning_org_unit_id uuid,
  UNIQUE(building_code, room_code)
);

CREATE TABLE section_meetings (
  id uuid PRIMARY KEY,
  section_id uuid NOT NULL REFERENCES sections(id),
  day_of_week smallint NOT NULL CHECK (day_of_week BETWEEN 1 AND 7),
  start_time time NOT NULL,
  end_time time NOT NULL,
  room_id uuid REFERENCES rooms(id),
  instructor_id uuid,
  date_start date NOT NULL,
  date_end date NOT NULL
);
```

Workflow schema

```
CREATE TABLE workflow_instances (
  id uuid PRIMARY KEY,
  entity_type text NOT NULL CHECK (entity_type IN ('section','calendar_event')),
  entity_id uuid NOT NULL,
  current_state text NOT NULL,
  initiated_by uuid NOT NULL,
  started_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE workflow_tasks (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  state text NOT NULL,
  assigned_role text NOT NULL,
  assigned_scope_id uuid,
  assigned_user_id uuid,
  status text NOT NULL CHECK (status IN ('open','approved','returned','denied','cancelled')),
  due_at timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  resolved_at timestamptz
);

CREATE TABLE workflow_transitions (
  id uuid PRIMARY KEY,
  workflow_instance_id uuid NOT NULL REFERENCES workflow_instances(id),
  from_state text NOT NULL,
  to_state text NOT NULL,
  action text NOT NULL,
  acted_by uuid NOT NULL,
  acted_as_role text NOT NULL,
  justification text,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Conflict schema

```
CREATE TABLE scheduling_conflicts (
  id uuid PRIMARY KEY,
  term_id uuid NOT NULL REFERENCES terms(id),
  conflict_type text NOT NULL CHECK (
    conflict_type IN (
      'instructor_overlap','room_overlap','capacity_mismatch',
      'pattern_violation','contact_hour_mismatch','feature_mismatch'
    )
  ),
  severity text NOT NULL CHECK (severity IN ('warning','blocking')),
  left_section_id uuid NOT NULL REFERENCES sections(id),
  right_section_id uuid REFERENCES sections(id),
  room_id uuid REFERENCES rooms(id),
  instructor_id uuid,
  status text NOT NULL CHECK (status IN ('open','waived','resolved')),
  waiver_reason text,
  waived_by uuid,
  created_at timestamptz NOT NULL DEFAULT now(),
  resolved_at timestamptz
);
```

RBAC schema

```
CREATE TABLE roles (
  id uuid PRIMARY KEY,
  code text UNIQUE NOT NULL,
  name text NOT NULL
);

CREATE TABLE permissions (
  id uuid PRIMARY KEY,
  resource text NOT NULL,
  action text NOT NULL,
  UNIQUE(resource, action)
);

CREATE TABLE role_permissions (
  role_id uuid NOT NULL REFERENCES roles(id),
  permission_id uuid NOT NULL REFERENCES permissions(id),
  PRIMARY KEY (role_id, permission_id)
);

CREATE TABLE role_bindings (
  id uuid PRIMARY KEY,
  user_id uuid NOT NULL,
  role_id uuid NOT NULL REFERENCES roles(id),
  scope_type text NOT NULL CHECK (scope_type IN ('institution','campus','college','department','room_po
  scope_id uuid NOT NULL,
  starts_at timestamptz,
  ends_at timestamptz,
  granted_by uuid NOT NULL,
  justification text
);
```

Audit schema

```
CREATE TABLE audit_events (
  id uuid PRIMARY KEY,
  actor_user_id uuid NOT NULL,
  acted_as_role text NOT NULL,
  module text NOT NULL,
  action text NOT NULL,
  target_type text NOT NULL,
  target_id uuid NOT NULL,
  scope_type text NOT NULL,
  scope_id uuid NOT NULL,
  reason_code text,
  metadata jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Endpoint-to-state mapping

The simplest way to think about the workflow API is by state transitions.

Endpoint	Allowed current state	Resulting state
`POST /sections/{id}/submit`	`draft`, `validation_failed`	`validation_pending`
`POST /workflow-tasks/{id}/approve`	`chair_review`, `dean_review`, `registrar_review`, `academic_affairs_review`	Next configured state
`POST /workflow-tasks/{id}/return`	Review states	`draft`
`POST /workflow-tasks/{id}/route-academic-affairs`	`validation_failed`, `chair_review`, `dean_review`, `registrar_review`	`academic_affairs_review`
`POST /sections/{id}/assign-room`	`registrar_review`, `room_coordination`	`room_coordination` or `approved_for_publish`
`POST /terms/{termId}/publish`	`approved_for_publish`	`published`
`POST /sections/{id}/lock`	`published`	`locked`
`POST /sections/{id}/request-change`	`published`	`change_requested`

Implementation guidance

Three implementation principles matter most:

- Keep workflow state on the section, but record every transition in append-only workflow tables.
- Resolve permissions from role templates plus scope bindings, not hard-coded user flags.
- Separate validation/conflict records from approvals so the system can distinguish technical failures from human decisions.

That design matches how registrar and Academic Affairs workflows are commonly structured: departments build, approvers route, registrars coordinate compliance and rooms, and special cases escalate to higher academic authority before final publish.[4] [2] [1]



1. https://public.powerdms.com/CSUChico/documents/2348080
2. https://fsw.teamdynamix.com/TDClient/3443/COP/KB/PrintArticle?ID=170404
3. https://registrar.tulane.edu/sites/default/files/2023-11/fall-2024-class-scheduling-timeline.pdf
4. https://www.aacrao.org/news/electronic-workflows-to-radically-support-graduate-education/
5. https://www.stmarytx.edu/policies/academic-affairs/class-scheduling-policy/
6. https://it.osu.edu/student-information-system-sis/sis-room-class-scheduling/1-course-class-workflow
7. https://registrar.calpoly.edu/academic-scheduling
8. https://registrar.uiowa.edu/faculty-and-staff/classroom-scheduling/course-scheduling-policies-and-procedures
9. https://coursedog.com/articles/best-practices-for-strategic-scheduling
10. https://registrar.vanderbilt.edu/workflow.php
11. https://registrar.utah.edu/scheduling/classes/scheduler-onboarding.php
12. https://bluepes.com/blog/role-based-access-education-systems
13. https://www.getinsight.space/resources/ferpa-compliance-software-guide
14. https://openeducat.org/compliance/ferpa/
15. https://www.coursera.org/
16. https://coursedog.my.site.com/support/s/article/WORKFLOWS-Creating-Editing-Duplicating-Deleting-Workflows
17. https://decisions.com/blog/three-workflows-to-automate-in-the-registrars-office